

Week 13 – Installing and Updating Software Packages

Topic covered: RH124 Chapter 12 – Install and Update Software Packages.

This week focuses on:

- Understanding RPM package names
- Querying installed and uninstalled RPM packages
- Verifying package scripts and signatures
- Installing and removing packages with `rpm`
- Managing dependencies with YUM or DNF
- Searching, installing, updating and removing packages
- Working with package groups
- Viewing and configuring software repositories
- Creating a local repository from a CentOS installation ISO

The uploaded summary identifies the practical as **RH124 Chapter 12.9: Lab Install and Update Software Packages**. It does not reproduce the complete official lab question sheet, so the practical workflow below follows the procedures taught in the Week 13 slides.

1. Understanding RPM packages

RPM stands for **RPM Package Manager**. Red Hat-based Linux distributions use RPM packages to distribute and manage software.

Two common RPM formats are:

Format	Meaning
<code>.rpm</code>	Compiled binary package ready for installation
<code>.src.rpm</code>	Source code used to build a binary RPM

A typical package filename has this structure:

`package-version-release.architecture.rpm`

Example from the slide:

x3270-x11-3.3.6-10.5.el6.i686.rpm

Section	Value
Package name	x3270-x11
Version	3.3.6
Release	10.5.el6
Architecture	i686

Common architectures include:

Architecture	Meaning
noarch	Does not contain architecture-specific binary code
src	Source package
i686	32-bit Intel-compatible package
x86_64	64-bit Intel or AMD package

2. Checking whether a package is installed

Use:

```
rpm -q package_name
```

Example:

```
rpm -q httpd
```

If installed, the output contains the complete package name:

```
httpd-2.4.57-...
```

If not installed:

```
package httpd is not installed
```

Check another package:

```
rpm -q bash
```

The `-q` option means **query**.

3. Listing all installed packages

```
rpm -qa
```

Meaning:

- `-q` – query
- `-a` – all installed packages

Display packages alphabetically:

```
rpm -qa | sort
```

Search for packages containing a keyword:

```
rpm -qa | grep httpd
```

The slides also use a wildcard query:

```
rpm -qa 'python*'
```

Display packages according to installation time:

```
rpm -qa --last
```

This lists the most recently installed packages first.

4. Displaying package information

For an installed package:

```
rpm -qi package_name
```

Example:

```
rpm -qi bash
```

This can display:

- Name
- Version
- Release
- Architecture
- Installation date
- Size
- Licence
- Vendor
- Description

For an RPM file that is not installed:

```
rpm -qip package_file.rpm
```

Example from the slides:

```
rpm -qip /media/Packages/httpd-2.4.6-88.el7.x86_64.rpm
```

The important difference is:

Target

Query form

Installed package	<code>rpm -qi package</code>
----------------------	----------------------------------

Uninstalled RPM file	<code>rpm -qip file.rpm</code>
-------------------------	------------------------------------

The `-p` option tells RPM to inspect a package file.

5. Useful RPM queries

Command	Purpose
<code>rpm -qa</code>	List all installed packages
<code>rpm -qi PACKAGE</code>	Display package information
<code>rpm -ql PACKAGE</code>	List files installed by the package
<code>rpm -qc PACKAGE</code>	List package configuration files
<code>rpm -qd PACKAGE</code>	List package documentation
<code>rpm -qR PACKAGE</code>	Show package requirements

<code>rpm -q --provides PACKAGE</code>	Show capabilities provided
<code>rpm -q --scripts PACKAGE</code>	Show installation and removal scripts
<code>rpm -qs PACKAGE</code>	Show the state of package files

For an uninstalled RPM file, add `p`:

1. `rpm -qpl package.rpm`
2. `rpm -qpR package.rpm`

`rpm -qp --scripts package.rpm`

6. Finding which package owns a file

Use:

`rpm -qf /path/to/file`

Example:

`rpm -qf /usr/bin/ls`

Possible result:

`coreutils-...`

This means `/usr/bin/ls` was installed by the `coreutils` package.

Another example:

`rpm -qf /etc/passwd`

This helps when you know a file or command but do not know its package name.

7. Listing files belonging to a package

```
rpm -ql package_name
```

Example:

```
rpm -ql bash
```

List only configuration files:

```
rpm -qc bash
```

List documentation:

```
rpm -qd bash
```

This is useful when you need to locate:

- Configuration files
 - Executable commands
 - Manuals
 - Documentation directories
-

8. Checking package dependencies

Check what an installed package requires:

```
rpm -qR package_name
```

Example:

```
rpm -qR httpd
```

Check the requirements of an RPM file:

```
rpm -qpR package_file.rpm
```

The output may include:

- Other packages
- Shared libraries
- Commands
- Configuration files
- Version requirements

9. Checking package scripts

Packages may execute scripts before or after installation or removal.

Inspect an installed package:

```
rpm -q --scripts package_name
```

Inspect an uninstalled package file:

```
rpm -qp --scripts package_file.rpm
```

Example from the slides:

```
rpm -qp --scripts /media/Packages/httpd-2.4.6-88.el7.x86_64.rpm
```

This may show:

- Pre-installation scripts
- Post-installation scripts
- Pre-uninstallation scripts
- Post-uninstallation scripts

The slides warn that third-party RPM scripts should be inspected because installation scripts run with root privileges and could damage the system.

10. Importing RPM signing keys

Trusted signing keys are normally stored under:

```
/etc/pki/rpm-gpg/
```

Import the available keys:

```
sudo rpm --import /etc/pki/rpm-gpg/*
```

List imported keys:

```
rpm -qa 'gpg-pubkey*'
```

Signing keys allow RPM to verify that a package came from a recognised source and was not altered after signing.

11. Verifying an RPM file

Use:

```
rpm -K package_file.rpm
```

Example:

```
rpm -K /media/Packages/httpd-2.4.6-88.el7.x86_64.rpm
```

A successful result may end with:

```
pgp md5 OK
```

The exact algorithms shown depend on the RPM and system version.

The Week 13 slides instruct users to import the relevant keys before performing this integrity verification.

12. Verifying installed package files

Use:

```
rpm -V package_name
```

Example:

```
rpm -V bash
```

If there is no output, the checked files generally match the RPM database records.

Verify all installed packages:

```
sudo rpm -Va
```

This may produce substantial output because configuration files and system changes commonly differ from their original package state.

Do not confuse:

Command

Purpose

<code>rpm -K</code>	Verify an RPM package file and its signature
<code>file.rpm</code>	

<code>rpm -V</code>	Verify installed files against RPM database
<code>package</code>	records

13. Installing packages using RPM

Basic command:

```
sudo rpm -i package_file.rpm
```

Commonly used verbose form:

```
sudo rpm -ivh package_file.rpm
```

Options:

Option	Meaning
<code>-i</code>	Install
<code>-v</code>	Verbose output
<code>-h</code>	Display progress using hash marks

Example:

```
sudo rpm -ivh example-package.rpm
```

Verify:

```
rpm -q example-package
```

14. RPM dependency problems

RPM does not normally download or install missing dependencies automatically.

For example:

```
sudo rpm -ivh httpd-2.4.6-88.el7.x86_64.rpm
```

may fail because `httpd` requires packages or files such as:

- `apr`
- `apr-util`
- `/etc/mime.types`
- Required shared libraries

The administrator must identify and install the required packages.

Several package files can be installed together:

```
sudo rpm -ivh apr.rpm apr-util.rpm mailcap.rpm httpd.rpm
```

The slides use this approach to resolve dependencies for an RPM installation.

This is one reason DNF or YUM is generally preferred: it resolves dependencies automatically.

15. Upgrading an RPM package

Use:

```
sudo rpm -Uvh package_file.rpm
```

The uppercase `-U` means upgrade.

It can:

- Upgrade an existing package
- Install the package if an older version is not installed

Another option is:

```
sudo rpm -Fvh package_file.rpm
```

-F, or **freshen**, upgrades the package only when an older version is already installed.

16. Removing packages using RPM

Use:

```
sudo rpm -e package_name
```

Example:

```
sudo rpm -e httpd
```

Use the package name rather than the RPM filename.

RPM prevents removal when another installed package depends on it.

Several related packages may need to be removed together:

```
sudo rpm -e httpd httpd-tools
```

The Week 13 slides note that dependencies can prevent both RPM installation and RPM removal.

17. Why YUM or DNF is preferred

YUM and DNF use software repositories and automatically:

- Locate packages
- Download package files
- Resolve dependencies
- Install required supporting packages
- Remove packages and unneeded dependencies
- Update packages
- Manage package groups

Modern Red Hat-based distributions generally use `dnf`. In many environments, `yum` is retained as a compatible command.

For these notes:

`dnf ...`

may often be replaced with:

`yum ...`

The slides show both YUM commands and DNF alternatives.

18. Listing repositories

`dnf repolist`

or:

`yum repolist`

List enabled and disabled repositories:

`dnf repolist --all`

Display repository information:

`dnf repoinfo`

A repository is a software source containing:

- RPM packages
- Package metadata
- Dependency information
- Package groups

19. Searching for packages

Search package names and descriptions:

`dnf search keyword`

Example:

```
dnf search web server
```

Search for `httpd`:

```
dnf search httpd
```

Display detailed package information:

```
dnf info httpd
```

Check whether it is installed:

```
dnf list installed httpd
```

List available packages:

```
dnf list available
```

20. Finding which package provides a command or file

Find the package providing a path:

```
dnf provides /usr/bin/command
```

Example:

```
dnf provides /usr/bin/httpd
```

A wildcard may be used when the complete path is unknown:

```
dnf provides '*/httpd'
```

This is the repository-based equivalent of investigating which package supplies a required file or command.

21. Installing packages with DNF

Install a package:

```
sudo dnf install package_name
```

Example:

```
sudo dnf install httpd
```

DNF displays:

- Package to install
- Dependencies
- Download size
- Installed size

It then asks for confirmation.

Automatically answer yes:

```
sudo dnf install httpd -y
```

Verify:

```
rpm -q httpd
```

or:

```
dnf list installed httpd
```

22. Removing packages with DNF

```
sudo dnf remove package_name
```

Example:

```
sudo dnf remove httpd
```

Automatically confirm:

```
sudo dnf remove httpd -y
```

DNF shows which related packages will be removed before making changes.

Verify:

```
rpm -q httpd
```

Expected:

```
package httpd is not installed
```

23. Updating packages

Check for available updates:

```
dnf check-update
```

Update one package:

```
sudo dnf update package_name
```

Example:

```
sudo dnf update bash
```

Update all packages:

```
sudo dnf update
```

On newer DNF systems, this may also be written as:

```
sudo dnf upgrade
```

Before updating all packages in a real server environment, the administrator should consider service impact and whether a reboot may be required.

24. Viewing DNF history

List package-management transactions:

```
dnf history
```

View one transaction:

```
dnf history info TRANSACTION_ID
```

Example:

```
dnf history info 5
```

Depending on the system and transaction, DNF may support undoing a transaction:

```
sudo dnf history undo 5
```

This history feature is useful for identifying when packages were installed, updated or removed.

25. Package groups

Package groups allow related packages to be managed together.

List groups:

```
dnf group list
```

Display information about a group:

```
dnf group info "Group Name"
```

Install a group:

```
sudo dnf group install "Group Name"
```

Example shown conceptually in the slides:

```
sudo yum groupinstall "Group Name"
```

Remove a group:

```
sudo dnf group remove "Group Name"
```

Group names containing spaces should be placed in quotation marks.

26. Repository configuration files

Repository files are stored in:

```
/etc/yum.repos.d/
```

They normally end with:

```
.repo
```

List them:

```
ls -l /etc/yum.repos.d/
```

View one:

```
cat /etc/yum.repos.d/example.repo
```

Common repository fields include:

3. [repository-id]
4. name=Repository description
5. baseurl=repository-location
6. enabled=1
7. gpgcheck=1

gpgkey=file:///path/to/key

Field	Purpose
[repository-id]	Unique repository identifier
name	Human-readable description
baseurl	Location of repository content
enabled=1	Enables the repository
enabled=0	Disables the repository
gpgcheck=1	Verifies package signatures
gpgkey	Location of the trusted signing key

27. Enabling and disabling repositories

Display all repositories:

```
dnf repolist --all
```

Temporarily use only one repository for a command:

```
dnf --disablerepo='*' --enablerepo=REPO_ID list available
```

Enable a repository persistently when the config-manager plugin is available:

```
sudo dnf config-manager --set-enabled REPO_ID
```

Disable it:

```
sudo dnf config-manager --set-disabled REPO_ID
```

The exact repository identifiers must be obtained using:

```
dnf repolist --all
```

8.